

Experiment 4

Advanced Prompting Techniques

Submission PDF

Source folder on GitHub

Includes the separate course item: [Ex04b_Scenario_Based_Report](#)

Prepared 2026-09-29 from the experiment materials on branch
``codex/complete-experiments``.

Source: [Ex04_Advanced_Prompting_Techniques/README.md](#)

Ex-4: Advanced Prompt Engineering Techniques

Due: 21 Aug 2026 **Reference repo:** RaajaThilahar/Ex.No.4

Techniques (7) x Case studies (3)

Techniques: Zero-shot, Few-shot, Chain-of-Thought, Persona, Reverse prompting, Graph prompting, Active prompting. Case studies: Smart Irrigation, Drone Navigation, Robot Path Planning.

Prompts live in `prompts/<technique>.md`, each with one prompt per case study. Run them, paste results to `outputs/`, and score in `evaluation.md`.

Evaluation metrics

Reasoning quality Correctness Token usage (record input + output tokens or word count as proxy).

Source: [Ex04_Advanced_Prompting_Techniques/outputs/README.md](#)

Save as `<technique>_<case>_<tool>.md`, e.g. `cot_irrigation_claude.md`. Record token counts at the top of each file.

Source: [Ex04b_Scenario_Based_Report/README.md](#)

Ex-4 (T2 track): Scenario-Based Report Development Using Diverse Prompting Techniques

Due: 25 May 2026 **Reference:**

Hemavathymanigandan/Ex-4.-Scenario-Based-Report-Development-Utilizing-Diverse-Prompting-Techniques

Objective

Understand a scenario and generate the prompts that produce a complete report.

Scenario (edit if your instructor gave another)

A city water board reports rising pipe leaks. Produce a management report recommending an AI-based leak-detection programme.

Files

- `scenario.md` - scenario analysis (stakeholders, goal, constraints, audience)
- `prompts/` - one prompt per report section, using a different technique each
- `outputs/` - model responses
- `report.md` - assembled final report

Source: [Ex04b_Scenario_Based_Report/outputs/README.md](#)

Paste each model response as `<section>_<tool>.md`.

Source: Ex04_Advanced_Prompting_Techniques/LAB_RECORD.md

Lab Record: Exp-4: Advanced Prompt Engineering Techniques

Field	Entry
Name / Reg. No.	_____
Date	_____
Repo	https://github.com/RHUDHRESH/PROMPT_ENGINEERING-

Aim

Implement zero-shot, few-shot, CoT, persona, reverse, graph and active prompting on engineering case studies.

Tools / Apparatus

Claude outputs cover the seven techniques; Grok (Fast) browser runs cover the Active Prompting samples only. ChatGPT and other platform runs for the other techniques were not performed.

Procedure

Prompts written (see prompts/ or the folder README).

Claude outputs are under outputs/. Grok ran five independent baseline and five exemplar-guided samples for each Active Prompting case; see outputs/active_grok.md.

Results scored in evaluation.md on: Reasoning, correctness, token usage.

Findings summarised below.

Observations

See evaluation.md and outputs/active_grok.md. Claude scores are self-assessments; Grok active-prompting results report observed decisions and are not expert scores.

Result

Seven techniques were applied to irrigation, drone navigation and robot path planning. Checkable calculations and the Dijkstra graph answer were compared with hand-worked ground truths; token values are word-count proxies.

Conclusion

Graph prompting made relationships and paths easy to inspect; few-shot prompts were concise for choices with clear examples. The Claude Active Prompting rows are simulations; separate Grok browser samples were run five times before and after a human-written exemplar for each case.

Faculty signature: _____

Source: Ex04b_Scenario_Based_Report/LAB_RECORD.md

Lab Record: Ex-4 (T2): Scenario-Based Report Development

Field	Entry
Name / Reg. No.	_____
Date	_____
Repo	https://github.com/RHUDHRESH/PROMPT_ENGINEERING-

Aim

Understand a scenario and generate prompts that build a full report.

Tools / Apparatus

Claude (done)

Procedure

Prompts written (see prompts/ or the folder README).

Each prompt run on every tool; raw outputs saved under outputs/ (or the folder's equivalent).

Results scored in report .md on: Report completeness and prompt-technique fit.

Findings summarised below.

Observations

See report .md. Add your own scores for any tool you ran yourself. Tools not run are left blank; do not fill them in from memory.

Result

Five section prompts and outputs were assembled into a board report recommending a six-month leak-detection pilot. Scenario facts are separated from illustrative assumptions.

Conclusion

Breaking a report into prompt sections supports clearer structure and review. Replace illustrative estimates with local water audits, supplier quotes and privacy approval before using the report to make a real funding decision.

Faculty signature: _____

Source: Ex04_Advanced_Prompting_Techniques/evaluation.md

Evaluation

Score Reasoning and Correctness 1-5. The main technique table reports Claude outputs; its scores are self-assessments by the model that wrote them. Grok (Fast) was used only for the additional Active Prompting run set below. ChatGPT and other platforms were not run for the remaining techniques. Treat model scores as provisional and re-score by hand.

Tokens column: real token counts were not available. The figure is an **approximate word count** (prompt + output, prompt estimated at about 45 words), rounded to the nearest 10. It is a proxy only; actual tokens are usually about 1.3x the word count for English text.

The three Claude Active rows below are **simulations**; see outputs/active_claude.md. Grok browser samples were run five times per case before and after exemplars; see outputs/active_grok.md.

Technique	Case	Tool	Reasoning	Correctness	Tokens (approx. words)	Notes
Zero-shot	Irrigation	Claude	3	4	~230	Reasonable pseudocode with tunable thresholds (illustrative values); no calibration.
Few-shot	Irrigation	Claude	3	4	~110	WATER (short); the label is a judgement between examples. Cheapest.
CoT	Irrigation	Claude	5	5	~120	$8000 - 2000 = 6000$ L; $6000/30 = 200$ min, matches ground truth.
Persona	Irrigation	Claude	4	4	~290	Practical and plain, budget-aware; numbers are rules of thumb.
Reverse	Irrigation	Claude	4	5	~190	Six well-targeted questions covering crop, water, soil, budget, power, goals.
Graph	Irrigation	Claude	4	5	~270	Adjacency list plus SPOF analysis (controller, pump/valve, sensor).
Active	Irrigation	Claude	4	4	~360	Simulated. Most uncertain of the three (3:2). Exemplar gives "do not skip, reduced dose".
Zero-shot	Drone	Claude	3	4	~280	Sound VIO/tracker /avoidance pipeline; notes monocular scale ambiguity.
Few-shot	Drone	Claude	3	4	~120	Yaw 30 deg right, slow, pass behind; action is a judgement, no single correct answer.
CoT	Drone	Claude	5	5	~210	$3 + 4.5 = 7.5$ m < 30 m; flagged the prompt's 2.5 s as inconsistent and bounded it (15 m).

Technique	Case	Tool	Reasoning	Correctness	Tokens (approx. words)	Notes
Persona	Drone	Claude	5	4	~290	Strong failure-mode list (spoofing, altitude error, dropout) and fixes.
Reverse	Drone	Claude	4	4	~240	Reconstructed prompt plus context list; inherently speculative.
Graph	Drone	Claude	4	5	~260	Mermaid state machine; no unreachable states, dead ends and a livelock loop identified.
Active	Drone	Claude	4	4	~450	Simulated. Exemplar makes wind/airspeed assumptions explicit; assumed 12 m/s airspeed is illustrative. Most tokens.
Zero-shot	Robot	Claude	4	4	~170	A* with Manhattan on 20x20 with justification.
Few-shot	Robot	Claude	4	5	~90	D* Lite; correct for known-but-changing grid. Cheapest.
CoT	Robot	Claude	5	5	~320	Included the tie-breaking caveat: with Manhattan and arbitrary ties, A* can still expand the whole rectangle.
Persona	Robot	Claude	4	5	~260	Correctly identifies BFS ignores weights; full-credit path Dijkstra/A/D Lite.
Reverse	Robot	Claude	3	3	~240	The "pasted design document" does not exist; used a stand-in (A* on 20x20) and said so.
Graph	Robot	Claude	5	5	~310	Full distance table; shortest path A,C,B,D,E,F = 13 (verified).
Active	Robot	Claude	4	4	~350	Simulated. Answer "stop then replan"; least disagreement (4:1) in simulation.

Grok active-prompting sample

Grok Fast was sampled in the browser for drone navigation, smart irrigation and robot path planning. Each case had five baseline answer variants, a human-written exemplar, and five exemplar-guided variants. Decisions stayed unanimous in every set: drone RETURN 5/5 before and after; irrigation WATER 5/5 before and after; robot REPLAN 5/5 before and after. The exemplars did not change binary decision consistency, but they prompted more cautious explanations for the drone and irrigation cases. See [`outputs/active\\_grok.md`](#) for prompts, exemplars and observations. This is not an expert accuracy score; the drone prompt lacks airspeed and energy-consumption inputs, and the irrigation prompt lacks crop-specific thresholds.

Ground truths to check against (verified)

- **CoT irrigation:** $2000 \text{ m}^2 \times 4 \text{ mm} = 8000 \text{ L}$ (1 mm over 1 $\text{m}^2 = 1 \text{ L}$); minus 1 mm rain (2000 L) = 6000 L ; $6000/30 = 200 \text{ min}$. Verified correct.
- **CoT drone:** distance = $6 \times 0.5 \text{ (delay)} + 62/(2 \times 4) = 3 + 4.5 = 7.5 \text{ m} < 30 \text{ m} \rightarrow \text{stops in time}$. Verified correct.
Correction to the earlier note: the old note called "2.5 s" a distractor and said stop time was 1.5 s of braking. The calculation is right (braking time $6/4 = 1.5 \text{ s}$), but the prompt's "needs 2.5 s to stop" is not a harmless distractor: it is inconsistent with the other numbers, since $0.5 \text{ s delay} + 1.5 \text{ s braking} = 2.0 \text{ s total}$, not 2.5 s. The answer does not depend on it: even at constant 6 m/s for the whole 2.5 s the drone covers at most 15 m ($< 30 \text{ m}$). Good answers should use $v^2/(2a)$ and state the inconsistency. (If instead 2.5 s were the braking time alone, deceleration would be $6/2.5 = 2.4 \text{ m/s}^2$, not 4 m/s^2 , so the prompt over-specifies.)
- **Dijkstra A->F:** verified. Distances: C=2, B=3 (via C), D=8 (via B), E=10 (via D), F=13 (via E); path A, C, B, D, E, F = 13. The other candidate A, C, B, D, F = 14. The old note listed the path as "A-C(2)-B(3)-D(8)-E(10)-F(13)", where the numbers are cumulative distances (edge weights are 2, 1, 5, 2, 3); this is correct but easily misread as edge weights.

Conclusion

Best technique per case (from these outputs, self-assessed):

- **Irrigation:** Chain-of-Thought for the calculation (exact, checkable, cheap at ~120 words). For design questions, Graph prompting gave the most useful structure (critical path and single points of failure), and Reverse prompting is the best first step when requirements are unknown.
- **Drone:** Chain-of-Thought for the stopping-distance question because it exposed the inconsistent 2.5 s figure; Persona (flight-controller engineer) for safety review; Graph (state machine) for mission-logic verification.
- **Robot:** Graph prompting with Dijkstra was exact and fully verifiable (13); CoT was best for the *A versus Dijkstra comparison*, and *Few-shot was the cheapest correct choice of algorithm* (D Lite, ~90 words).

Token trade-off: Few-shot (~90-120 words) and CoT numeric problems (~120) were the cheapest per useful answer. Persona, Graph and Reverse cost about 200-300 words and repay it when the task needs breadth, structure or review. Active prompting requires five samples, an exemplar and five re-runs. In the Grok browser runs below, all three binary decisions were already unanimous before the exemplar, so the exemplar did not improve decision consistency; it made the drone and irrigation rationales more cautious. The Claude Active rows remain simulations. Overall rule: use CoT for calculations, Few-shot for classification or choices with clear precedents, Persona/Graph for design and safety analysis, Reverse for requirements gathering, and reserve Active prompting for places where repeated runs really do disagree.

Caveats: Claude scores are self-scored, word counts are proxies for tokens, and the Grok active-prompting observations are single-session samples with some truncated answers. The Claude Active rows are simulated.

Source: Ex04b_Scenario_Based_Report/report.md

AI Leak Detection Pilot - Board Report

Prepared for the Board of Directors. Decision requested: approve a 6-month pilot. Draft written by Claude from the scenario in scenario.md; sections were produced with different prompting techniques (see prompts/ and outputs/).

> **Facts vs assumptions.** The only facts taken from the scenario are: rising pipe leaks, about 30% non-revenue water, a 6-month pilot, a limited budget and a data-privacy constraint. Everything else marked "illustrative" (the split of the 30% by cause, option ratings, roles and timings) is an assumption for discussion and must be replaced with the water board's own data and supplier quotes before a funding decision.

1. Executive summary

Roughly 30% of the water we treat is lost before it earns revenue, and rising pipe leaks are a growing part of that. Today we find leaks mainly by manual surveys and customer reports, so many run for weeks unseen. We propose a six-month, limited-budget pilot of AI-based leak detection in one or two selected zones. Machine-learning models will analyse pressure and flow data to flag likely leaks, and crews will verify and repair them. The pilot will show whether the approach finds leaks faster, cuts water losses and pays back, without exposing customer data. We ask the Board to approve the pilot and to judge it against clear targets at month six before deciding on wider rollout.

2. Problem analysis

Step 1 - what non-revenue water (NRW) is. NRW is water put into the network that is never billed. It is made up of (a) real losses (physical leakage and overflows), (b) apparent losses (water used but not measured or billed) and (c) unbilled authorised use (flushing, firefighting).

Step 2 - causes and estimated share of the 30%. The scenario gives only the 30% total. The split below is an **illustrative assumption** based on typical utility water balances, not this Board's data; it should be replaced by the water board's own water audit.

Rank	Cause	Category	Assumed share of NRW	Approx. points of the 30%
1	Leaks on mains and service connections (ageing pipes, pressure surges)	Real loss	60%	18
2	Customer-meter under-registration	Apparent loss	17%	5
3	Illegal connections and theft	Apparent loss	10%	3
4	Billing and data-handling errors	Apparent loss	7%	2
5	Unbilled authorised use (flushing, firefighting)	Unbilled authorised	6%	2

Step 3 - what AI can address.

- **Leakage (rank 1): strongly.** Models on pressure and flow data can spot the signature of new leaks early and narrow the search area, so crews go to the right place sooner. Faster detection also cuts the running time of each leak.
- **Meter under-registration and theft (ranks 2-3): partly.** Consumption-pattern anomaly detection can flag suspect connections, but this is outside the pilot's core scope.
- **Billing errors and unbilled authorised use (ranks 4-5): little.** These are process problems, better fixed by data clean-up and policy.

Conclusion: if the illustrative split is roughly right, leakage is the largest and most AI-addressable cause, so a pilot focused on leak detection targets the biggest share of the problem. Even a modest improvement there would be material.

3. Options compared

The ratings are **qualitative and illustrative**, based on general industry experience, because no local quotes or trial data were provided. Costs are relative, not dollar figures; real values need supplier quotes.

Option	Cost	Accuracy	Time to deploy	Data needed	Risk
Manual survey (crews with listening equipment)	Low equipment cost, high ongoing labour	Medium; depends on crew skill, mostly finds existing leaks	Immediate (weeks)	None beyond network maps	Slow and periodic; limited coverage; leaks run between surveys
Acoustic sensors (loggers on valves and hydrants)	Medium to high (hardware, installation, maintenance)	High for locating leaks on metal pipes; weaker on plastic pipes	2-4 months for a zone	Acoustic signals; pipe material and map	Sensor drift, battery life, vandalism; poor performance on plastic pipe
Satellite imagery	Medium per survey; no field hardware	Low to medium; indicates suspect areas, not exact points	Weeks to a few months	Imagery plus network map	Coarse resolution, false positives, weather; best as a screening tool
ML on pressure/flow data	Low to medium if existing zone meters and SCADA are used; higher if new meters are needed	Medium to high for detecting a leak in a zone; needs field work to pinpoint	2-4 months (data audit, training, validation)	Historic and live pressure and flow, GIS, repair records	Data quality, false alarms, model drift, privacy if meter-level data is used

Reading the table: no option is best on every criterion. ML on pressure data is cheapest per area covered and best for early detection, while acoustic loggers are the best way to pinpoint the leak once a zone is flagged. Manual crews remain essential for verification.

4. Risks and mitigations

The two examples from the prompt are kept, followed by five new risks.

- Risk: Sensor drift -> Impact: false alarms -> Mitigation: monthly calibration.
- Risk: Poor data quality -> Impact: model misses leaks -> Mitigation: data audit before pilot.
- Risk: Customer data privacy breach (meter-level consumption reveals household behaviour) -> Impact: legal exposure and loss of public trust -> Mitigation: use zone-level pressure/flow data, anonymise or aggregate meter data, apply access controls and a privacy assessment before go-live.
- Risk: Too many false positives -> Impact: crews waste time on dry holes and lose confidence in the tool -> Mitigation: agree a target precision with crews, tune alert thresholds in month 2-3, and report false-alarm rate weekly.
- Risk: Pilot zones not representative of the whole network -> Impact: results overstate or understate real benefit, leading to a wrong rollout decision -> Mitigation: choose zones with mixed pipe age and materials and record their characteristics with the results.
- Risk: Repair crews cannot keep up with detected leaks -> Impact: savings are not realised and detected leaks keep running -> Mitigation: reserve crew capacity for the pilot zones and prioritise repairs by estimated loss.
- Risk: Vendor lock-in or cost overrun -> Impact: budget exceeded or data trapped in one platform -> Mitigation: fixed-price pilot contract, agreed data-export terms and a stage gate at month three.

5. Recommendation

We recommend that the Board approve a six-month pilot of ML leak detection on pressure and flow data in one or two zones, using acoustic loggers and crews to pinpoint and verify leaks.

- **Run a data audit and select pilot zones.** Owner: Head of Network Operations. Deadline: end of month 1.
- **Build and validate the model, with privacy sign-off.** Owner: Data and IT lead with the Data Protection Officer. Deadline: end of month 3 (stage gate).
- **Verify alerts in the field, repair leaks and report results.** Owner: Pilot project manager. Deadline: month 6 Board meeting.

Expected savings: if the pilot finds and repairs leaks earlier, it should reduce the leakage share of the 30% non-revenue water in the pilot zones, with the size of the saving to be measured against the baseline established in month 1.

Appendix: assumptions and open items

- The cause split of non-revenue water (60/17/10/7/6%) is illustrative, not measured.
- Option costs, accuracy and deployment times are qualitative industry-style ratings; no quotes, dollar values or trial results were available.
- Pilot budget, zone selection, owners' job titles and the month deadlines are proposals, not existing commitments.
- Expected savings are deliberately not quantified; the month-1 baseline will set the measurable target.

Source:
Ex04_Advanced_Prompting_Techniques/outputs/active_claude.md

Active prompting - Claude outputs (3 case studies)

Tool: Claude, answered directly. Token counts below are approximate word counts (prompt + output).

IMPORTANT - THIS IS A SIMULATION. Active prompting needs 5 real independent samples per question (fresh runs with sampling temperature) and a human-written exemplar. I cannot run 5 independent samples of myself inside one response, and this repository has no logs of such runs. So for each case I reasoned about which parts of the question are ambiguous and where a model's answers would plausibly diverge, and wrote 5 "samples" that represent that reasoning. The sample counts (e.g., 4 RETURN / 1 CONTINUE) are my estimate of the likely spread, not measured frequencies. The exemplars are written by me (Claude) as a stand-in for the human-written exemplar the method calls for, and the "after exemplar" consistency is a projection, not an experiment. Treat all agreement numbers as illustrative; re-run with real samples before drawing conclusions.

Case 1: Drone (continue or return)

Approx. words (prompt + output): about 450

Step 1 - simulated 5 samples Question: 20 min battery, 1 kg payload, 6 km from base, 8 m/s headwind. CONTINUE or RETURN? Where answers would vary: the prompt gives no airspeed, no remaining mission distance, and no direction for the headwind (on the way out or the way home?). Samples would differ on these assumptions.

#	Answer	One-line reason (as a sample would write it)
1	RETURN	Headwind and payload cut endurance, and 6 km with 20 min left leaves too little reserve.
2	RETURN	Assuming about 12 m/s airspeed, a headwind leaves about 4 m/s over the ground, so the distance takes too long.
3	CONTINUE	Assumes the headwind helps on the return leg and the mission target is near.
4	RETURN	Standard 20-30% battery reserve rule is not met.

#	Answer	One-line reason (as a sample would write it)
5	RETURN	Headwind uncertainty and payload make continuing unsafe.

Simulated agreement: 4 of 5 for RETURN. Disagreement is small on the final label but large in the reasoning, because each sample assumes a different airspeed and wind direction. That is the uncertainty to fix.

Step 2 - exemplar (written by Claude in place of a human)

Q: A drone with 20 min battery, 1 kg payload, 6 km from base and an 8 m/s headwind: continue or return?
Reasoning: The prompt has no airspeed or wind direction, so use the worst case. Assume cruise airspeed 12 m/s (illustrative). If the headwind is on the way home, ground speed is $12 - 8 = 4$ m/s, so $6000 \text{ m} / 4 = 1500 \text{ s} = 25 \text{ min}$, which is more than the 20 min battery, and the payload increases power draw. If the headwind is on the way out, the return has a tailwind: $6000 / 20 = 300 \text{ s} = 5 \text{ min}$, plenty of margin. Continuing means flying further away, which increases the return distance. Rule: if the energy needed to get home with a 25% reserve cannot be shown to be available under the worst-case wind, return now.
A: RETURN (now, and land early if the battery cannot cover the trip home)

Step 3 - simulated re-run with exemplar: with the worst-case rule shown, all 5 samples would be expected to say RETURN and to give the same energy-reserve reasoning (projected 5/5, not measured).

Case 2: Irrigation (skip watering?)

Approx. words (prompt + output): about 360

Step 1 - simulated 5 samples Question: Skip watering if 3 mm rain is forecast but soil is at 12%? Where answers would vary: whether 3 mm of rain is enough to matter (it is small), whether the forecast is reliable, and whether 12% is critically dry for this soil (no soil type given).

#	Answer	Reason
1	Do not skip	12% is very dry and 3 mm is a small amount.
2	Skip	Rain is coming, so avoid over-watering; check again tomorrow.
3	Do not skip	Forecast rain is uncertain and light rain gets caught by leaves.
4	Skip	Assumes rain will arrive soon and the crop can wait a few hours.
5	Do not skip	Soil is far below the typical 25-30% threshold, so water a reduced amount.

Simulated agreement: 3 vs 2, the largest disagreement among the three questions in this exercise, so it is the best candidate for an exemplar.

Step 2 - exemplar (written by Claude)

Q: Skip watering if 3 mm rain is forecast but soil is at 12%?
Reasoning: 3 mm on 1 m² is 3 L, a small amount compared with a typical daily crop demand of 4-6 mm. Soil at 12% is far below the usual 25-30% threshold, so the deficit is large and is not closed by 3 mm. The forecast may not arrive or may be intercepted by the canopy. So do not skip; water for the deficit minus the forecast rain (about 3 mm credit), and re-check soil moisture after the rain.
A: DO NOT SKIP - water a reduced dose (deficit minus 3 mm)

Step 3 - simulated re-run: projected 5/5 "do not skip, reduced dose". Not measured.

Case 3: Path planning (replan or continue?)

Approx. words (prompt + output): about 340

Step 1 - simulated 5 samples Question: Replan or continue when a new obstacle appears 1 cell ahead? Where answers would vary: whether the obstacle is static or moving, and whether "replan" means a full replan or a local repair.

#	Answer	Reason
1	REPLAN	An obstacle directly ahead blocks the current path.
2	REPLAN	Stop and replan locally to avoid a collision.
3	REPLAN	The stored path is now invalid.
4	CONTINUE	Assumes the obstacle is moving and will clear the cell.
5	REPLAN	Use incremental repair (D* Lite).

Simulated agreement: 4 of 5 REPLAN; the variation comes from the unstated static-versus-moving assumption.

Step 2 - exemplar (written by Claude)

Q: Replan or continue when a new obstacle appears 1 cell ahead?

Reasoning: With one cell of clearance the robot cannot wait for a prediction. If the next cell is occupied, continuing along the current path causes a collision, whether the obstacle is static or moving, so first stop (or hold). Then check the path: if the cell is blocked, the current path is invalid. Repair it with incremental search (D* Lite) and resume. If the obstacle is known to be moving away, waiting a step is acceptable, but stopping is always safe.

A: STOP, then REPLAN (incremental repair)

Step 3 - simulated re-run: projected 5/5 "stop then replan". Not measured.

Summary of this simulation

Most uncertain: irrigation (3:2), then drone (4:1 label, divergent reasoning), then path planning (4:1). The exemplar mainly works by making the missing assumption explicit (worst-case wind, soil deficit versus rain, stopping first).

Cost: exemplars add roughly 100-150 words of prompt each.

Source:

[Ex04_Advanced_Prompting_Techniques/outputs/active_grok.md](#)

Active Prompting: Grok Browser Runs

Date: 2026-09-29. Tool: Grok Fast in the user's existing Edge session. Each baseline prompt was submitted once and regenerated to collect five answer variants. The exemplar prompt was then submitted as a new conversation and regenerated until five variants were collected. Decisions below are transcribed from the visible browser results; incomplete response endings are marked. Exact model seed and token counts were not exposed.

Drone navigation

Baseline prompt: Should a drone with 20 min battery, carrying a 1 kg payload, 6 km from base and facing 8 m/s headwind, continue the mission or return? Answer CONTINUE or RETURN and give a one-line reason.

Five baseline decisions: RETURN, RETURN, RETURN, RETURN, RETURN (5/5). The rationales varied: headwind and payload reduce the energy margin; a return may consume the remaining endurance; one answer assumed the return leg would have a tailwind. One response ended mid-sentence. These assumptions cannot be resolved from the prompt because airspeed, power draw, wind direction along the full route, and safety reserve are missing.

Human-written exemplar:

> Q: Should a drone with 20 min battery, carrying a 1 kg payload, 6 km from base and facing 8 m/s headwind, continue the mission or return? > Reasoning: The prompt omits airspeed, battery consumption under load, wind variability and required reserve, so a safe round-trip margin cannot be verified. Use the conservative failsafe when the remaining-energy margin is unknown. > A: RETURN.

Five exemplar-guided decisions: RETURN, RETURN, RETURN, RETURN, RETURN (5/5). Reasons consistently noted that the missing energy inputs prevent verifying a safe return margin.

Smart irrigation

Baseline prompt: Skip watering if 3 mm rain is forecast but soil is at 12%? Answer WATER or SKIP and give a one-line reason.

Five baseline decisions: WATER, WATER, WATER, WATER, WATER (5/5). Rationales said the forecast rain may be too light or uncertain to relieve low soil moisture; two responses visibly ended mid-sentence.

Human-written exemplar:

> Q: Skip watering if 3 mm rain is forecast but soil is at 12%? > Reasoning: Check the crop-specific moisture trigger, soil, rain timing and sensor reliability. If 12% is below the validated trigger, a short calibrated dose followed by a recheck is safer than skipping solely because 3 mm is forecast. > A: WATER.

Five exemplar-guided decisions: WATER, WATER, WATER, WATER, WATER (5/5). The responses repeated the need for a short dose/recheck and noted that the forecast may not reach the root zone.

Robot path planning

Baseline prompt: Replan or continue when a new obstacle appears 1 cell ahead? Answer REPLAN or CONTINUE and give a one-line reason.

Five baseline decisions: REPLAN, REPLAN, REPLAN, REPLAN, REPLAN (5/5). Reasons consistently said the immediate path is blocked, so continuing risks collision and the map/path should be updated.

Human-written exemplar:

> Q: Replan or continue when a new obstacle appears 1 cell ahead? > Reasoning: The next move would enter a newly occupied cell. Stop before it, update the map and compute a collision-free path that respects robot clearance. > A: REPLAN.

Five exemplar-guided decisions: REPLAN, REPLAN, REPLAN, REPLAN, REPLAN (5/5). Exemplar-guided reasons retained the stop, map update and safe-path steps.

Observation and limits

All three decisions were already unanimous at baseline, so the exemplar did not improve binary decision consistency (5/5 before and after). It made rationales more cautious and specific for the drone and irrigation cases. These are single-model, single-session observations; repeated generations may share hidden system context, and regeneration variants are not independent laboratory trials. Grok searched the web for some answers, response latency and token counts were not recorded, and some rationales were cut off in the interface. Check engineering decisions against domain data before use.

Source:

[Ex04_Advanced_Prompting_Techniques/outputs/cot_claude.md](#)

Chain of Thought - Claude outputs (3 case studies)

Tool: Claude, answered directly. Token counts below are approximate word counts (prompt + output).

Case 1: Smart Irrigation

Approx. words (prompt + output): about 120

Water needed: 4 mm over 1 m² is 4 L, so 2000 m² x 4 mm = 8000 L.

Rain credit: 1 mm over 2000 m² = 2000 L.

Remaining: $8000 - 2000 = 6000$ L.

Pump time: $6000 \text{ L} / 30 \text{ L/min} = 200$ min.

Answer: 200 minutes (3 h 20 min). Assumes the forecast rain is fully effective, and no losses.

Case 2: Drone Navigation

Approx. words (prompt + output): about 210

Given: $v = 6 \text{ m/s}$, obstacle 30 m ahead, detection delay 0.5 s, deceleration 4 m/s^2 .

Distance during the delay (no braking yet): $6 \times 0.5 = 3$ m.

Braking time: $6 / 4 = 1.5$ s.

Braking distance: $v^2/(2a) = 36 / 8 = 4.5$ m.

Total stopping distance: $3 + 4.5 = 7.5$ m, which is less than 30 m, so **the drone stops in time**, with a 22.5 m margin.

Inconsistency in the prompt: "needs 2.5 s to stop" does not match the numbers. Delay plus braking is $0.5 + 1.5 = 2.0$ s in total, not 2.5 s. I used the physics (4 m/s^2 from 6 m/s) rather than the 2.5 s. Even if the 2.5 s were true and the drone flew at full speed for the entire 2.5 s, it would cover at most $6 \times 2.5 = 15$ m, still under 30 m. So the conclusion holds either way.

Case 3: Robot Path Planning

Approx. words (prompt + output): about 320

Setup: 100×100 grid = 10,000 cells, 4-connectivity, every step costs 1. Worst-case start (0,0) to goal (99,99): shortest length $d = 198$ steps.

Dijkstra with uniform cost behaves like BFS. It expands nodes in order of distance from the start, so it expands every cell with distance $< d$, a diamond that here covers essentially the entire grid: about 10,000 expansions in the worst case (about half the grid for a goal in the middle). It has no direction knowledge.

A with Manhattan heuristic $h = |dx| + |dy|$ is admissible and consistent for 4-connectivity, so it is still optimal. It expands only nodes with $f = g + h \leq d$. In an obstacle-free grid every monotone path has $f = d$ exactly, so if ties are broken arbitrarily A* may still expand up to the whole rectangle between start and goal (up to about 10,000 cells in this worst case). With tie-breaking toward larger g (or a tiny heuristic weight), it expands about the path length, roughly 200 nodes. With obstacles, A* expands more, but still typically far fewer than Dijkstra.*

Effect of the heuristic: it reorients the search toward the goal. The price is a small per-node overhead (computing h and a priority queue), so the gain is largest when the goal is far and the map is fairly open. If the heuristic were 0, A* equals Dijkstra.

****Recommendation:** A* with Manhattan distance and larger- g tie-breaking. ** Same optimal path, far fewer expansions. Dijkstra is only preferable if costs are unknown or there are many goals (one search from the start reaches all).

Source:

[Ex04_Advanced_Prompting_Techniques/outputs/few_shot_claude.md](#)

Few-shot - Claude outputs (3 case studies)

Tool: Claude, answered directly. Token counts below are approximate word counts (prompt + output).

Case 1: Smart Irrigation

Approx. words (prompt + output): about 110

Input: moisture=22%, rain=1mm, temp=31C -> Decision: **WATER (short)**

Reason: moisture is low-ish and it is hot, while 1 mm of rain is negligible. It is not as dry as the 15% example (long), and the rain is far below the 8 mm example (skip). The exact label "short" extrapolates between the examples; it is a judgement call.

Case 2: Drone Navigation

Approx. words (prompt + output): about 120

Obstacle 4m ahead, moving across path right-to-left -> ACTION: **yaw 30 right, slow, pass behind the obstacle's trailing side, then resume heading**

Reason: the obstacle is leaving the path toward the left, so passing on the right moves away from where it is heading. It is not moving toward the drone, so a climb is unnecessary. Speed is reduced because it is only 4 m away.

Case 3: Robot Path Planning

Approx. words (prompt + output): about 90

Grid 50x50, dynamic obstacles, replanning needed -> ****D* Lite****

Reason: the grid is known but changes, and D\ *Lite repairs the previous search incrementally instead of replanning from scratch (unlike A\)*, following the pattern of the "unknown map" example.

Source:

[Ex04_Advanced_Prompting_Techniques/outputs/graph_claude.md](#)

Graph prompting - Claude outputs (3 case studies)

Tool: Claude, answered directly. Token counts below are approximate word counts (prompt + output).

Case 1: Smart Irrigation

Approx. words (prompt + output): about 270

Adjacency list (directed; data/control flows):

Soil sensor	-> Controller	(moisture readings)
Weather API	-> Controller	(rain forecast, temperature)
Controller	-> Valve	(open/close command)
Controller	-> Pump	(start/stop command)
Controller	-> Farmer app	(status, alerts)
Farmer app	-> Controller	(manual override, settings)
Pump	-> Valve	(water flow / pressure; physical link)
Valve	-> (field)	(sink: water delivered)

Critical path: Soil sensor -> Controller -> (Pump + Valve) -> water in the field. Weather API is an input to the same decision. The action needs both Pump and Valve, so both sit on the critical path.

Single points of failure:

- **Controller:** every data and control edge passes through it. If it fails, nothing is watered (and nothing can be overridden).
- **Pump:** no pump means no flow even if the valve opens. **Valve** likewise, per zone.
- **Soil sensor** (if there is only one): the controller becomes blind and must fall back to schedules.
- **Not failures of the automation:** Weather API (the controller can decide from soil moisture alone) and Farmer app (used for oversight and override, not the control loop), though losing the app removes visibility.
- Not modelled but critical: power and network.

Mitigations: watchdog and a safe default schedule in the controller, redundant sensors, manual valve bypass, pressure sensing on the pump.

Case 2: Drone Navigation

Approx. words (prompt + output): about 260

```
stateDiagram-v2
    [*] --> Takeoff
    Takeoff --> Cruise: target altitude reached
    Takeoff --> Emergency: motor/sensor fault
    Cruise --> Avoid: obstacle within safety distance
    Cruise --> Land: destination reached
    Cruise --> Emergency: low battery / link loss / sensor failure
    Avoid --> Cruise: obstacle cleared
    Avoid --> Reroute: obstacle persists, path blocked
    Avoid --> Emergency: collision imminent / fault
    Reroute --> Cruise: new path found
    Reroute --> Emergency: no feasible path / battery critical
    Emergency --> Land: safe landing spot found or immediate descent
    Land --> [*]
```

Unreachable states: none. Every state can be reached from Takeoff.

Dead ends and design issues:

- **Land** is a terminal state (dead end by design); that is the intended end of the mission.
- **Emergency** has only one exit (Land). There is no path back to Cruise, which is intentional but means any false alarm aborts the mission. Consider a "recovered" transition for transient faults.
- **Avoid <-> Reroute <-> Cruise** can cycle forever (livelock) if the obstacle keeps blocking; add a timeout that sends the drone to Emergency or Land.
- Takeoff has no "abort to Land" edge for non-fault reasons (e.g., bad weather), and Land has no failure edge (e.g., landing zone blocked). These are missing transitions rather than unreachable states.

Case 3: Robot Path Planning

Approx. words (prompt + output): about 310

Graph (undirected): A-B:4, A-C:2, B-C:1, B-D:5, C-D:8, C-E:10, D-E:2, D-F:6, E-F:3. Source A, target F.

Step	Node settled (dist)	A	B	C	D	E	F	Note
0	init	0	inf	inf	inf	inf	inf	
1	A (0)	0*	4	2	inf	inf	inf	relax A-B (4), A-C (2)
2	C (2)	0*	3	2*	10	12	inf	B improves 4 -> 3 via C; D=2+8=10; E=2+10=12
3	B (3)	0*	3*	2*	8	12	inf	D improves 10 -> 8 via B (3+5)
4	D (8)	0*	3*	2*	8*	10	14	E improves 12 -> 10 via D (8+2); F=8+6=14
5	E (10)	0*	3*	2*	8*	10*	13	F improves 14 -> 13 via E (10+3)
6	F (13)	0*	3*	2*	8*	10*	13*	target settled

(* means settled/final.)

Predecessors: F <- E <- D <- B <- C <- A.

Shortest path A -> F: A, C, B, D, E, F, with total cost 13. Check: $2 + 1 + 5 + 2 + 3 = 13$. Alternative A-C-B-D-F costs $2+1+5+6 = 14$, so it is worse by 1.

Source:

[Ex04_Advanced_Prompting_Techniques/outputs/persona_claude.md](#)

Persona pattern - Claude outputs (3 case studies)

Tool: Claude, answered directly, role-playing the persona in each prompt. Token counts below are approximate word counts (prompt + output).

Case 1: Smart Irrigation (agronomist, limited budget)

Approx. words (prompt + output): about 290

Plain advice for tomatoes on a small budget:

Sensors (cheap first):

- Two or three soil-moisture probes, one for each part of the plot that behaves differently (sandy corner, low wet spot, main bed), put at root depth, about 15-20 cm. Capacitive ones last longer than the cheapest resistive ones, which corrode.
- A simple rain gauge (a straight-sided jar works) instead of a weather service.
- Optional later: a low-cost timer/controller with a solenoid valve.
- Skip fancy sensors (EC, leaf wetness) until the basics pay off.

Schedule (adjust to your soil):

- Drip lines, early morning, so water is not lost to heat and leaves stay dry (less blight).
- Seedlings to first flowers: short, frequent watering to keep the soil evenly damp. Flowering and fruiting: deeper watering, roughly every 2-3 days in loam, more often in sand. Stay steady, because swings in moisture cause cracked fruit and blossom-end rot.
- Rule of thumb: water when the probe reads dry at root depth, or when the top 5 cm feels dry but 10 cm down is still cool and damp. Skip after 10 mm or more of rain. Cut back in cool cloudy weeks.
- Mulch (straw) saves a lot of water and costs almost nothing.

Start with hand valves and the probes; add the automatic timer once you trust the readings. (Any numbers here are rules of thumb, not measured values for your farm.)

Case 2: Drone Navigation (UAV flight-controller engineer)

Approx. words (prompt + output): about 290

Review of "Use GPS only for obstacle avoidance": **reject**. GPS tells you where you are, not what is in front of you.

Failure modes:

GPS knows nothing about obstacles that are not on a map: people, birds, wires, other drones, cranes, moving vehicles.

Position accuracy is metres, with a refresh of only a few Hz. That is too coarse and too slow for close avoidance.

Dropouts and degradation: urban canyons, trees, indoors, bridges, multipath, and jamming or spoofing. A spoofed drone will confidently fly into things.

Altitude error is worse than horizontal error, so vertical separation is unreliable.

Latency and an uncertain map: even with a map, out-of-date data (new construction) is a hazard.

No fail-safe behaviour defined when GPS is lost.

Fixes:

- Add perception: obstacle sensing with at least one, ideally two, independent modalities (stereo or depth camera, lidar or radar, ultrasonic/ToF for close range).
- Sensor fusion (EKF) with IMU, barometer and optical flow, so state estimation does not depend on GPS alone.
- Detect GPS quality (HDOP, satellite count, innovation checks) and spoofing plausibility checks.
- Geofence, altitude margins and a terrain/obstacle database as a prior only.
- Defined degraded modes: on GPS loss, switch to visual-inertial navigation, slow down, hold or land, then return-to-launch only if the path is validated.
- Verification: hardware-in-the-loop and failure-injection tests, and a conservative speed limit tied to sensing range (stopping distance below detection range).

Case 3: Robot Path Planning (robotics professor grading BFS plan)

Approx. words (prompt + output): about 260

Feedback (grade: partial credit). BFS is a sound algorithm for the right problem, but it is the wrong tool here. BFS minimises the number of steps, not the total cost. In a warehouse with weighted costs (congested aisles, turns, one-way lanes, travel through restricted zones), a route with the fewest cells can be expensive. Your plan would return a path that is short in hops but not in cost, and you should tell me you understand that.

What earns full credit:

State the objective explicitly: minimise a cost function (distance plus turn penalty plus congestion).

Use **Dijkstra** if you have no useful heuristic, or **A*** with an admissible heuristic (for example, Manhattan distance scaled by the minimum edge cost), which keeps optimality and reduces expansions.

If aisle costs or blocked cells change (other robots, pallets), use **D* Lite** to repair the plan incrementally.

Note the multi-robot issue (deadlocks, priorities or reservation tables) and dimension your grid resolution against the robot footprint (inflate obstacles).

Give a complexity comment ($O(E + V \log V)$ for Dijkstra with a heap) and a test with a case where BFS and Dijkstra disagree.

One small credit-worthy point: if all costs were uniform, BFS and Dijkstra would coincide, so BFS is fine there.

Source:

[Ex04_Advanced_Prompting_Techniques/outputs/reverse_claude.md](#)

Reverse prompting - Claude outputs (3 case studies)

Tool: Claude, answered directly. Token counts below are approximate word counts (prompt + output).

Case 1: Smart Irrigation (model asks questions, does not design yet)

Approx. words (prompt + output): about 190

What do you grow, and on how much land? (crop types, field size and layout, growth stage, whether zones differ)

What is your water source and current equipment? (well, canal, tank; pump size and flow rate; drip, sprinkler or flood; any existing valves or controllers)

What soil and climate conditions apply? (soil type, slope, typical rainfall and heat, how often you see drought or waterlogging)

What budget and maintenance capacity do you have? (upfront limit, running costs, who repairs it, how technical the user is)

What connectivity and power are available? (mains, solar, mobile signal or Wi-Fi, LoRa range; whether it must work offline)

What outcomes matter most and what limits apply? (save water, save labour, raise yield; water-use quotas or fertiliser (fertigation) needs; how much manual override you want and what alerts you need)

Case 2: Drone Navigation (write the prompt that would produce this output)

Approx. words (prompt + output): about 240

Target output: "Use a PID controller with obstacle repulsive potential fields."

Best prompt:

You are a control engineer. I have a quadcopter flying from A to B in an environment with obstacles. Recommend a control and avoidance approach, in one or two sentences, that combines a classical feedback controller for tracking with a reactive obstacle-avoidance method. Prefer well-established, lightweight methods that run on a small onboard computer. Answer concisely with the method names and how they connect.

****Context the prompt must include (otherwise the answer could just as well be MPC, RRT*, or a learned policy):****

- Vehicle: quadcopter, dynamics and rate of the inner loop, computing budget.
- Sensors available (range/depth, camera, GPS, IMU) and their update rates.
- Environment: static or moving obstacles, indoor or outdoor, density of clutter.
- Task and objective: point-to-point flight, with the safety margin and maximum speed.
- Desired answer style: short, name the algorithms, no derivations.
- Preferences and constraints: prefers classical/explainable methods, lightweight compute, reactive (no global map).
- Known caveat worth asking for: potential fields can get stuck in local minima, so ask the answer to mention it if that is wanted.

Case 3: Robot Path Planning (reconstruct the requirements prompt)

Approx. words (prompt + output): about 240

Limitation: the prompt says "Given this final design document (paste)", but no document was pasted in the repository. I did not invent one. As a stand-in, I used the design decision from Case 3 of the zero-shot and CoT outputs in this exercise (A* with Manhattan heuristic on a 20x20 warehouse grid with static shelves) and reconstructed the likely requirements prompt from it. It is only a plausible reconstruction.

Reconstructed prompt:

Design the path-planning module for a single warehouse robot operating on a 20x20 grid map with static shelves (no moving obstacles). The robot moves in 4 directions with uniform step cost. It must compute optimal (shortest) paths from any start to any pick location in real time on a small embedded computer. Choose an algorithm, justify it against alternatives (Dijkstra, D* Lite, RRT), specify the heuristic, state the expected node expansions, and note limitations and when the design would need to change (dynamic obstacles, multiple robots, weighted aisles).

Requirements that are inferable from such a document: grid size and connectivity, static environment, optimality, real-time limit, single robot, and explicit non-goals (multi-robot, dynamic obstacles).

Source:**Ex04_Advanced_Prompting_Techniques/outputs/zero_shot_claude.md**

Zero-shot - Claude outputs (3 case studies)

Tool: Claude, answered directly. Token counts below are approximate word counts (prompt + output), a proxy only.

Case 1: Smart Irrigation

Approx. words (prompt + output): about 230

```
# Inputs: moisture (%), rain_mm (forecast, next 24 h), temp_c
# Tunable constants
DRY = 25          # % below which the crop is stressed
WET = 45          # % above which watering is never needed
RAIN_SKIP = 5     # mm of forecast rain that is enough to skip
HOT = 30          # deg C

function decide(moisture, rain_mm, temp_c):
    if moisture >= WET:
        return SKIP
    if rain_mm >= RAIN_SKIP and moisture >= DRY:
        return SKIP          # rain will cover the need
    if moisture < DRY:
        minutes = base_minutes
        if temp_c >= HOT: minutes = minutes * 1.3  # more evaporation
        minutes = minutes - rain_mm * minutes_per_mm # credit forecast rain
        return WATER(max(minutes, min_run))
    # moderate zone: DRY <= moisture < WET
    if temp_c >= HOT and rain_mm < 1:
        return WATER(short)
    return SKIP

# Extra rules: never water within 1 h of measured rain, cap daily runtime,
# fall back to a time schedule if the moisture sensor value is missing or out of range.
```

The thresholds are illustrative and must be calibrated per soil and crop.

Case 2: Drone Navigation

Approx. words (prompt + output): about 280

Estimate own motion (IMU + camera). Fuse IMU (accelerometer and gyro) with visual odometry (feature tracking) in an EKF to get pose and velocity. Without GPS, the camera corrects IMU drift; the IMU covers gaps between frames.

Detect the obstacle. Run an object detector or optical-flow / motion segmentation on each frame. A monocular camera has no direct depth, so estimate range from the obstacle's apparent size growth (looming / time-to-contact) or from motion parallax as the drone moves.

Track and predict. Keep a Kalman-filter track of the obstacle's bearing and range and extrapolate a short horizon (1-3 s) to predict where it will be, with an uncertainty envelope.

Plan. Follow a straight-line or waypoint path A to B. When the predicted obstacle envelope crosses the path within the time-to-contact threshold, generate an avoidance manoeuvre (lateral or vertical offset away from the obstacle's motion, using a velocity-obstacle or potential-field method), then blend back to the A-B line.

Control. A cascaded PID/MPC controller turns the reference velocity into attitude commands, with speed limits so stopping distance is shorter than the detection range.

Safety. If tracking is lost or the estimate is uncertain, slow or hover, and use a simple rule such as "obstacle in view and growing means move away". The key limitation is scale ambiguity of a single camera, so use conservative margins.

Case 3: Robot Path Planning

Approx. words (prompt + output): about 170

****Choice: A*** with a Manhattan-distance heuristic (4-connected grid), on a precomputed occupancy grid.**

Why: a 20x20 grid has only 400 cells, static shelves mean the map never changes, and A* *is optimal (with an admissible heuristic) while expanding far fewer nodes than Dijkstra or BFS. Manhattan distance is admissible and consistent for 4-connectivity, so the first time the goal is popped the path is optimal. It is simple to implement and debug. Alternatives: Dijkstra is correct but explores more; D\ Lite only pays off when obstacles appear or move; RRT/PRM target continuous, high-dimensional spaces. Caveat: if aisles have costs (one-way lanes, congestion) or many robots, add cost weights and a multi-robot layer (for example, reservation tables).*

Source:

[Ex04_Advanced_Prompting_Techniques/prompts/01_zero_shot.md](#)

Zero-shot

Smart Irrigation

Design a rule for a smart irrigation controller that decides when to water using soil moisture (%), rain forecast (mm) and temperature (C). Give the rule as pseudocode.

Drone Navigation

Describe how a quadcopter should navigate from point A to B while avoiding a moving obstacle, using only a camera and IMU.

Robot Path Planning

Choose a path-planning algorithm for a warehouse robot on a 20x20 grid with static shelves and explain the choice.

Source:

[Ex04_Advanced_Prompting_Techniques/prompts/02_few_shot.md](#)

Few-shot

Smart Irrigation

Classify irrigation decisions.
Input: moisture=15%, rain=0mm, temp=34C -> Decision: WATER (long)
Input: moisture=55%, rain=0mm, temp=25C -> Decision: SKIP
Input: moisture=30%, rain=8mm, temp=28C -> Decision: SKIP (rain expected)
Input: moisture=22%, rain=1mm, temp=31C -> Decision:

Drone Navigation

Choose the avoidance action.
Obstacle 5m ahead, static -> ACTION: yaw 30 and pass
Obstacle 3m ahead, moving toward drone -> ACTION: climb 2m and slow
Obstacle 2m left, moving away -> ACTION: continue
Obstacle 4m ahead, moving across path right-to-left -> ACTION:

Robot Path Planning

Pick algorithm.
Grid 10x10, static, single goal -> A*
Unknown map, exploring -> D* Lite
Continuous space, high-dimensional arm -> RRT*
Grid 50x50, dynamic obstacles, replanning needed ->

Source: [Ex04_Advanced_Prompting_Techniques/prompts/03_chain_of_thought.md](#)

Chain of Thought

Smart Irrigation

A field of 2000 m² needs 4 mm of water/day. The pump delivers 30 L/min. Rain forecast is 1 mm. How many minutes must the pump run? Think step by step, show each calculation, then state the final answer.

Drone Navigation

A drone flies at 6 m/s toward an obstacle 30 m ahead and needs 2.5 s to stop with 4 m/s² braking after a 0.5 s detection delay. Will it stop in time? Reason step by step.

Robot Path Planning

Compare A* and Dijkstra on a 100x100 grid with 4-connectivity and uniform cost. Reason step by step about node expansions and the effect of the heuristic, then recommend one.

Source: [Ex04_Advanced_Prompting_Techniques/prompts/04_persona_pattern.md](#)

Persona pattern

Smart Irrigation

Act as an agronomist with 20 years of drip-irrigation experience advising a small farmer with a limited budget. Recommend a sensor set and watering schedule for tomatoes. Speak plainly.

Drone Navigation

Act as a UAV flight-controller engineer at a safety-critical company. Review this plan: "Use GPS only for obstacle avoidance." List the failure modes and fixes.

Robot Path Planning

Act as a senior robotics professor grading a student's plan to use BFS for a weighted-cost warehouse. Give feedback and a corrected approach.

Source: [Ex04_Advanced_Prompting_Techniques/prompts/05_reverse_prompting.md](#)

Reverse prompting (model asks / infers the prompt)

Smart Irrigation

I want a smart irrigation system for my farm. Do not answer yet. Ask me the 6 most important questions you need to design it, one list only.

Drone Navigation

Here is the output I got: "Use a PID controller with obstacle repulsive potential fields." Write the best prompt that would have produced exactly this kind of answer, and list what context the prompt must include.

Robot Path Planning

Given this final design document (paste), reconstruct the original requirements prompt that an engineer likely wrote.

Source: [Ex04_Advanced_Prompting_Techniques/prompts/06_graph_prompting.md](#)

Graph prompting

Smart Irrigation

Represent the irrigation system as a graph: nodes = {soil sensor, weather API, controller, valve, pump, farmer app}; edges = data/control flows. Output the adjacency list, then explain the critical path and single points of failure.

Drone Navigation

Model the drone mission as a directed graph of states (Takeoff, Cruise, Avoid, Reroute, Land, Emergency) with transition conditions on edges. Output as Mermaid, then list unreachable/dead-end states.

Robot Path Planning

Given the weighted graph: A-B:4, A-C:2, B-C:1, B-D:5, C-D:8, C-E:10, D-E:2, D-F:6, E-F:3. Find the shortest path A->F with Dijkstra, showing the distance table at each step.

Source: [Ex04_Advanced_Prompting_Techniques/prompts/07_active_prompting.md](#)

Active prompting (uncertainty-driven example selection)

Procedure: run the question 5 times, find the question with the most disagreement among answers, then have a human write a chain-of-thought exemplar for it and add it as a few-shot example.

Step 1 - sample (run 5x)

Should a drone with 20 min battery, carrying a 1 kg payload, 6 km from base and facing 8 m/s headwind, continue the mission or return? Answer CONTINUE or RETURN and give a one-line reason.

Step 2 - exemplar (human-written)

Q: <the most uncertain question>
Reasoning: <your step-by-step reasoning>
A: <correct answer>

Step 3 - re-run with the exemplar prepended and compare consistency across 5 runs. Repeat for irrigation ("Skip watering if 3 mm rain is forecast but soil is at 12%?") and path planning ("Replan or continue when a new obstacle appears 1 cell ahead?").

Source: [Ex04b_Scenario_Based_Report/outputs/1_executive_summary_claude.md](#)

Executive summary - role prompt - Claude output

Tool: Claude, answered directly (not ChatGPT). Text below the line is the section as used in report.md.

Roughly 30% of the water we treat is lost before it earns revenue, and rising pipe leaks are a growing part of that. Today we find leaks mainly by manual surveys and customer reports, so many run for weeks unseen. We propose a six-month, limited-budget pilot of AI-based leak detection in one or two selected zones. Machine-learning models will analyse pressure and flow data to flag likely leaks, and crews will verify and repair them. The pilot will show whether the approach finds leaks faster, cuts water losses and pays back, without exposing customer data. We ask the Board to approve the pilot and to judge it against clear targets at month six before deciding on wider rollout.

Source: [Ex04b_Scenario_Based_Report/outputs/2_problem_analysis_claude.md](#)

Problem analysis - chain of thought - Claude output

Tool: Claude, answered directly (not ChatGPT). Text below the line is the section as used in report.md.

Step 1 - what non-revenue water (NRW) is. NRW is water put into the network that is never billed. It is made up of (a) real losses (physical leakage and overflows), (b) apparent losses (water used but not measured or billed) and (c) unbilled authorised use (flushing, firefighting).

Step 2 - causes and estimated share of the 30%. The scenario gives only the 30% total. The split below is an **illustrative assumption** based on typical utility water balances, not this Board's data; it should be replaced by the water board's own water audit.

Rank	Cause	Category	Assumed share of NRW	Approx. points of the 30%
1	Leaks on mains and service connections (ageing pipes, pressure surges)	Real loss	60%	18
2	Customer-meter under-registration	Apparent loss	17%	5
3	Illegal connections and theft	Apparent loss	10%	3
4	Billing and data-handling errors	Apparent loss	7%	2
5	Unbilled authorised use (flushing, firefighting)	Unbilled authorised	6%	2

Step 3 - what AI can address.

- **Leakage (rank 1): strongly.** Models on pressure and flow data can spot the signature of new leaks early and narrow the search area, so crews go to the right place sooner. Faster detection also cuts the running time of each leak.
- **Meter under-registration and theft (ranks 2-3): partly.** Consumption-pattern anomaly detection can flag suspect connections, but this is outside the pilot's core scope.
- **Billing errors and unbilled authorised use (ranks 4-5): little.** These are process problems, better fixed by data clean-up and policy.

Conclusion: if the illustrative split is roughly right, leakage is the largest and most AI-addressable cause, so a pilot focused on leak detection targets the biggest share of the problem. Even a modest improvement there would be material.

Source: [Ex04b_Scenario_Based_Report/outputs/3_options_table_claude.md](#)

Options comparison - tabular prompt - Claude output

Tool: Claude, answered directly (not ChatGPT). Text below the line is the section as used in report.md.

The ratings are **qualitative and illustrative**, based on general industry experience, because no local quotes or trial data were provided. Costs are relative, not dollar figures; real values need supplier quotes.

Option	Cost	Accuracy	Time to deploy	Data needed	Risk
Manual survey (crews with listening equipment)	Low equipment cost, high ongoing labour	Medium; depends on crew skill, mostly finds existing leaks	Immediate (weeks)	None beyond network maps	Slow and periodic; limited coverage; leaks run between surveys
Acoustic sensors (loggers on valves and hydrants)	Medium to high (hardware, installation, maintenance)	High for locating leaks on metal pipes; weaker on plastic pipes	2-4 months for a zone	Acoustic signals; pipe material and map	Sensor drift, battery life, vandalism; poor performance on plastic pipe
Satellite imagery	Medium per survey; no field hardware	Low to medium; indicates suspect areas, not exact points	Weeks to a few months	Imagery plus network map	Coarse resolution, false positives, weather; best as a screening tool
ML on pressure/flow data	Low to medium if existing zone meters and SCADA are used; higher if new meters are needed	Medium to high for detecting a leak in a zone; needs field work to pinpoint	2-4 months (data audit, training, validation)	Historic and live pressure and flow, GIS, repair records	Data quality, false alarms, model drift, privacy if meter-level data is used

Reading the table: no option is best on every criterion. ML on pressure data is cheapest per area covered and best for early detection, while acoustic loggers are the best way to pinpoint the leak once a zone is flagged. Manual crews remain essential for verification.

Source: [Ex04b_Scenario_Based_Report/outputs/4_risks_claude.md](#)

Risks and mitigations - few-shot prompt - Claude output

Tool: Claude, answered directly (not ChatGPT). Text below the line is the section as used in report.md.

The two examples from the prompt are kept, followed by five new risks.

- Risk: Sensor drift -> Impact: false alarms -> Mitigation: monthly calibration.
- Risk: Poor data quality -> Impact: model misses leaks -> Mitigation: data audit before pilot.
- Risk: Customer data privacy breach (meter-level consumption reveals household behaviour) -> Impact: legal exposure and loss of public trust -> Mitigation: use zone-level pressure/flow data, anonymise or aggregate meter data, apply access controls and a privacy assessment before go-live.
- Risk: Too many false positives -> Impact: crews waste time on dry holes and lose confidence in the tool -> Mitigation: agree a target precision with crews, tune alert thresholds in month 2-3, and report false-alarm rate weekly.
- Risk: Pilot zones not representative of the whole network -> Impact: results overstate or understate real benefit, leading to a wrong rollout decision -> Mitigation: choose zones with mixed pipe age and materials and record their characteristics with the results.
- Risk: Repair crews cannot keep up with detected leaks -> Impact: savings are not realised and detected leaks keep running -> Mitigation: reserve crew capacity for the pilot zones and prioritise repairs by estimated loss.
- Risk: Vendor lock-in or cost overrun -> Impact: budget exceeded or data trapped in one platform -> Mitigation: fixed-price pilot contract, agreed data-export terms and a stage gate at month three.

Source:**Ex04b_Scenario_Based_Report/outputs/5_recommendation_claude.md****Recommendation - constraint plus output format - Claude output**

Tool: Claude, answered directly (not ChatGPT). Text below the line is the section as used in report.md.

We recommend that the Board approve a six-month pilot of ML leak detection on pressure and flow data in one or two zones, using acoustic loggers and crews to pinpoint and verify leaks.

- **Run a data audit and select pilot zones.** Owner: Head of Network Operations. Deadline: end of month 1.
- **Build and validate the model, with privacy sign-off.** Owner: Data and IT lead with the Data Protection Officer. Deadline: end of month 3 (stage gate).
- **Verify alerts in the field, repair leaks and report results.** Owner: Pilot project manager. Deadline: month 6 Board meeting.

Expected savings: if the pilot finds and repairs leaks earlier, it should reduce the leakage share of the 30% non-revenue water in the pilot zones, with the size of the saving to be measured against the baseline established in month 1.

(Approx. 140 words, under the 150 limit.)

Source: Ex04b_Scenario_Based_Report/prompts/1_executive_summary_role.md**Technique: Role prompt**

You are a utilities consultant. Write a 120-word executive summary for a water board on adopting AI leak detection. Scenario: ~30% non-revenue water, 6-month pilot, limited budget.

Source:**Ex04b_Scenario_Based_Report/prompts/2_problem_analysis_cot.md****Technique: Chain of thought**

Analyse step by step why non-revenue water reaches 30%: list causes, estimate relative contribution, and rank them. Then state which causes AI can address.

Source: Ex04b_Scenario_Based_Report/prompts/3_options_table.md**Technique: Tabular**

Compare 4 leak-detection options (manual survey, acoustic sensors, satellite, ML on pressure data) in a table: Cost | Accuracy | Time to deploy | Data needed | Risk.

Source: Ex04b_Scenario_Based_Report/prompts/4_risks_few_shot.md

Technique: Few-shot

Write risks in this style:
Risk: Sensor drift -> Impact: false alarms -> Mitigation: monthly calibration.
Risk: Poor data quality -> Impact: model misses leaks -> Mitigation: data audit before pilot.
Now write 5 more risks for the AI leak-detection pilot.

Source: Ex04b_Scenario_Based_Report/prompts/5_recommendation_constraints.md

Technique: Constraint + output format

Write the recommendation section: max 150 words, three bullet next-steps with owner and deadline, one sentence on expected savings. Use only facts from the previous sections.

Source: Ex04b_Scenario_Based_Report/scenario.md

Scenario analysis

Item	Answer
Situation	Rising pipe leaks; ~30% non-revenue water
Reader	Board of directors (non-technical)
Goal	Decide whether to fund a pilot
Constraints	6-month pilot, limited budget, data privacy
Deliverable	2-page report + recommendation
Success criteria	Clear cost/benefit, risks, next steps